ASSIGNMENT- 13.5

Name: K.Archana

HT.No: 2303A51329

Batch: 20

Task 1: Refactoring Global Variables

The legacy code uses a global variable `rate` that is accessed inside the function. Global variables reduce modularity and make testing difficult. The code should be refactored to pass the value as a parameter.

Prompt: Refactor the Python code to remove global variables and pass the required values as function parameters.

Code:

```python
#Task 1 – Refactoring Global Variables
def calculate_interest(amount, rate):
    return amount * rate


rate = 0.1
amount = 1000

interest = calculate_interest(amount, rate)
print("Interest:", interest)
```

Result:

```
Interest: 100.0
Very Good
```

Observation:

Passing the rate as a parameter removes dependency on global variables. This improves modularity, readability, and testability. The function can now be reused with different rates without modifying global state.

Task 2: Refactoring Deeply Nested Conditionals

The legacy code contains deeply nested conditional statements that reduce readability. The code should be simplified using cleaner conditional logic

Prompt: Refactor nested if–else conditions into a

cleaner and more readable structure.

Code :

```
#Task 2 – Refactoring Deeply Nested Conditionals
score = 78

if score >= 90:
    result = "Excellent"
elif score >= 75:
    result = "Very Good"
elif score >= 60:
    result = "Good"
else:
    result = "Needs Improvement"

print(result)
```

Result:

```
Interest: 100.0
Very Good
```

Observation:

Flattening nested conditions improves readability and maintainability. The refactored code is easier to understand and reduces unnecessary indentation.

Task 3: Refactoring Repeated File Handling Code

The legacy code repeatedly opens and closes files manually. This violates the DRY (Don't Repeat Yourself) principle and can lead to resource management issues.

Prompt: # Refactor repeated file open/read/close operations using a reusable function and context manager.

Code:

```python
#Task 3 – Refactoring Repeated File Handling Code
def read_file(filename):
    with open(filename, "r") as file:
        print(file.read())


read_file("data1.txt")
read_file("data2.txt")
```

Result:

```
Interest: 100.0
Very Good
Hello from data1 file
This is a test file.
Hello from data2 file
Python file handling test.
```

Observation:

Using a context manager (`with open`) ensures automatic file closing and reduces repetition. The reusable function improves maintainability and readability.

Task 4: Optimizing Search Logic

The legacy code performs a linear search through a list, which is inefficient for large datasets. It can be optimized using a set for faster lookup.

Prompt: # Optimize user search logic by using an appropriate data structure to improve

time complexity.

Code:

```
#Task 4 – Optimizing Search Logic
users = {"admin", "guest", "editor", "viewer"}

name = input("Enter username: ")

if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

Result:

```
Interest: 100.0
Very Good
Hello from data1 file
This is a test file.
Hello from data2 file
Python file handling test.
Enter username: admin
Access Granted
Net Salary: 40000.0
```

Observation

Using a set reduces search complexity from **O(n)** to **O(1)** on average. This significantly improves performance when the number of users increases.

Task 5: Refactoring Procedural Code into OOP

The legacy code calculates salary tax using procedural variables. It should be refactored into a class-based design using object-oriented principles.

Prompt: # Refactor the procedural salary calculation code into an object-oriented class with attributes and methods.

Code:

```python
#Task 5 - Refactoring Procedural Code into OOP

class EmployeeSalaryCalculator:

    def __init__(self, salary, tax_rate):
        self.salary = salary
        self.tax_rate = tax_rate

    def calculate_net_salary(self):
        tax = self.salary * self.tax_rate
        net_salary = self.salary - tax
        return net_salary


employee = EmployeeSalaryCalculator(50000, 0.2)

print("Net Salary:", employee.calculate_net_salary())
```

Result:

```
Very Good
Hello from data1 file
This is a test file.
Hello from data2 file
Python file handling test.
Enter username: admin
Access Granted
Net Salary: 40000.0
```

Observation:
Refactoring into a class improves encapsulation and scalability. It allows better organization of salary-related operations and makes the code easier to extend.

Task 6: Performance Optimization

The legacy code iterates through a large loop to calculate the sum of even numbers. This can be optimized using mathematical formulas.

Prompt: # Optimize the loop that calculates the sum of even numbers using a mathematical approach.

Code:

```
#Task 6 – Performance Optimization

n = 1000000

even_numbers = n // 2
total = even_numbers * (even_numbers + 1)

print("Sum of even numbers:", total)
```

Result:

```
Very Good
Hello from data1 file
This is a test file.
Hello from data2 file
Python file handling test.
Enter username: admin
Access Granted
Net Salary: 40000.0
Sum of even numbers: 250000500000
```

Observation:
The optimized version replaces an O(n) loop with an O(1) mathematical formula. This greatly improves performance for large datasets

Task 7: Removing Hidden Side Effects

The legacy code modifies a global list inside a function, creating hidden side effects. The function should return values instead.

Prompt: # Refactor the function to avoid

modifying global state and return updated values

instead.

Code:

```python
#Task 7 – Removing Hidden Side Effects
def add_item(data, value):
    new_data = data.copy()
    new_data.append(value)
    return new_data


data = []

data = add_item(data, 10)
data = add_item(data, 20)

print(data)
```

Result:

```
Interest: 100.0
Very Good
Hello from data1 file
This is a test file.
Hello from data2 file
Python file handling test.
Enter username: admin
Access Granted
Net Salary: 40000.0
Sum of even numbers: 250000500000
[10, 20]
```

Observation:

Returning new values instead of modifying global variables improves predictability and reduces side effects. This approach supports functional programming principles.

Task 8: Refactoring Complex Input Validation

The legacy code uses deeply nested conditions for password validation. The logic should be modularized into separate functions.

Prompt: # I Refactor password validation logic into separate functions to improve readability and maintainability.

Code:

```python
#Task 8 — Refactoring Complex Input Validation
def is_length_valid(password):
    return len(password) >= 8


def has_digit(password):
    return any(c.isdigit() for c in password)


def has_uppercase(password):
    return any(c.isupper() for c in password)


password = input("Enter password: ")

if not is_length_valid(password):
    print("Password too short")
elif not has_digit(password):
    print("Must contain digit")
elif not has_uppercase(password):
    print("Must contain uppercase")
else:
    print("Valid Password")
```

Result:

```
Interest: 100.0
Very Good
Hello from data1 file
This is a test file.
Hello from data2 file
Python file handling test.
Enter username: admin
Access Granted
Net Salary: 40000.0
Sum of even numbers: 250000500000
[10, 20]
Enter password: Vaishu@1234
Valid Password
```

Observation:
Breaking validation rules into separate functions improves readability and testability. Each function now

has a clear responsibility, making the code easier to maintain and extend.